Yes — that is a strong mode idea, but I would not call it “scrape mode.” I’d frame it as a **Research-Augmented Prompt Mode** so it searches, cites, and updates the prompt template responsibly instead of blindly scraping pages.

Thought for 20s ›

Yes. That should absolutely be a mode.

I would call it:

```
Research-Augmented Prompt Mode
```

or:

```
Web-Validated Template Mode
```

Not “scrape mode.” The app should **search and cite** sources, not blindly scrape the internet. OpenAI’s web search tooling is explicitly designed to let models access up-to-date internet information and return sourced citations, and OpenRouter also supports web-search plugins, including `:online` model variants and domain filters. OpenAI Platform OpenRouter

What this mode should do

The mode should take the user’s input, family, and active template, then run a **template improvement pass** before generating the final prompt.

The loop would be:

```
User input
→ classify family
→ load matching template
→ search web for best practices / current standards / missing constraints
→ compare sources against the template
→ recommend template tweaks
→ generate upgraded prompt
→ cite what influenced the changes
```

Example

User says:

```
Make a prompt for building an AI SaaS MVP with OpenRouter, Supabase, and Vercel.
```

Normal mode would generate the prompt from your existing `product_spec` or `execution_plan` template.

Research-Augmented Prompt Mode would first search for:

```
current OpenRouter API best practices
Supabase free-tier limitations
Vercel deployment constraints
AI SaaS MVP checklist
auth/database/security basics
prompting best practices for coding agents
```

Then it would say:

```
Template gap found:
- Current template does not ask for provider fallback behavior.
- Current template does not include rate-limit handling.
- Current template does not include environment-variable safety.
- Current template does not include deployment acceptance checks.
```

Then it modifies the generated prompt to include those missing pieces.

That is very valuable.

The mode config

I would add this as a built-in mode:

YAML

```yaml
mode_id: research_augmented_prompt
display_name: Research-Augmented Prompt
version: 1
status: built_in
mutable: false
copyable: true
default_prompt_family: analysis
token_policy: deep
scope_policy: evidence_first
question_policy: ask_blocking_only
critique_policy: required_before_accept
context_policy: web_sources_plus_user_context
loop_policy: semi_auto
connector_policy: web_search
```

Mode behavior

```
1. Classify the user request.
2. Identify the active prompt family.
3. Identify the active template.
4. Generate 3-7 research queries.
5. Search reliable sources.
6. Extract best practices, constraints, risks, and missing fields.
7. Compare source findings against the template.
8. Produce template tweak suggestions.
9. Generate the improved final prompt.
10. Cite source-backed changes.
```

The key feature: Template Gap Analysis

This should be its own reusable template.

```
prompt_library/template_gap_analysis/template_v1.md
```

It should ask:

```
Given:
- User input
- Prompt family
- Active mode
- Current template
- Web research notes

Find:
1. Missing context fields
2. Missing constraints
3. Missing output sections
4. Missing acceptance criteria
5. Missing safety/cost/privacy concerns
6. Missing tool/API/version assumptions
7. Better wording for the final prompt
8. Whether the template itself should be updated permanently
```

I would make it a separate loop

Call it:

```
Research → Compare → Improve Loop
```

Pass 1 — Search intent planner

```
Input:
User goal + family + current template

Output:
Search queries and source priorities
```

Pass 2 — Source collector

```
Input:
Search queries

Output:
Source notes with citations, reliability score, and relevance score
```

Pass 3 — Template gap analyzer

```
Input:
Current template + source notes

Output:
What the template is missing
```

Pass 4 — Prompt upgrader

```
Input:
User goal + current template + gap report

Output:
Improved canonical prompt
```

Pass 5 — Template update recommendation

```
Input:
Gap report + improved prompt

Output:
Temporary tweak or permanent template change recommendation
```

That last part matters. Not every web result should permanently change your template.

Template tweak decision system

The app should show a popup:

```
Research found 4 possible template improvements.

[Apply to this run only]
[Save as template suggestion]
[Update template permanently]
[Ignore]
```

Each recommendation should have:

```
Finding
Source
Confidence
Affected template section
Suggested change
Apply / Defer / Ignore
```

Web source rules

This mode needs strict source quality rules.

Source priority

```
1. Official docs
2. Standards / protocol specs
3. Maintainer docs
4. High-quality technical blogs
5. Recent tutorials
6. Community discussions
7. Reddit / forums only as weak signal
```

Search should be scoped by family

For example:

Family	Search source priority
`product_spec`	product docs, launch checklists, UX patterns
`execution_plan`	official setup docs, deployment docs
`debugging_troubleshooting`	GitHub issues, changelogs, official docs
`research_brief`	academic, government, primary sources
`outreach_persuasion`	platform rules, buyer psychology, examples

Family	Search source priority
`connector_spec`	API docs, auth docs, SDK docs

It should not blindly scrape websites

It should use **search APIs, official docs, provider search tools, or user-approved URL fetching** first.

If you later add true scraping/crawling, the app needs rules around robots.txt, rate limits, and permissions. Google's robots.txt documentation says robots.txt tells crawlers which URLs they can access, mainly to avoid overloading a site; it also warns robots.txt is not a security mechanism and not all crawlers obey it. Google for Dev…

So the safe product design is:

```
Search first.
Fetch only selected URLs.
Respect robots.txt.
Cache source summaries.
Cite every source-backed change.
Let the user approve permanent template updates.
```

What this adds to `prompt_library/`

I would add these folders:

```
prompt_library/
  research_augmented_prompt/
    template_v1.md
    search_planner_v1.md
    source_collector_v1.md
    template_gap_analysis_v1.md
    prompt_upgrade_v1.md
    permanent_template_update_recommendation_v1.md

  modes/
    research_augmented_prompt/
      template_v1.md

  wrappers/
    web_search_wrapper_v1.md
    openrouter_online_wrapper_v1.md
```

```
    source_citation_wrapper_v1.md

  evaluators/
    source_quality_evaluator_v1.md
    template_gap_evaluator_v1.md
    hallucination_risk_evaluator_v1.md
```

Suggested mode template

Markdown

```
---
mode_id: research_augmented_prompt
display_name: Research-Augmented Prompt
version: 1
status: built_in
mutable: false
copyable: true
default_prompt_family: analysis
token_policy: deep
scope_policy: evidence_first
question_policy: ask_blocking_only
critique_policy: required_before_accept
context_policy: web_sources_plus_user_context
loop_policy: semi_auto
connector_policy: web_search
---

# Research-Augmented Prompt Mode

## Purpose

Improve the final prompt by checking current, source-backed best practices before finaliz

## Required inputs

- User request
- Active prompt family
- Active mode
- Current template
- Available context pack
- Allowed web/source policy

## Process

1. Classify the user request and target family.
2. Identify what kind of sources would improve the prompt.
3. Generate focused search queries.
4. Retrieve source-backed notes.
```

```
5. Compare source findings against the current template.
6. Identify missing fields, constraints, risks, and acceptance checks.
7. Generate a better final prompt.
8. Cite which findings changed the prompt.
9. Recommend whether any template changes should be saved permanently.

## Output

Return:

1. Research query plan
2. Source notes
3. Template gap report
4. Suggested template tweaks
5. Improved final prompt
6. Permanent template update recommendation
7. Next prompt
```

UI feature

In the app, this should look like a toggle:

```
[ ] Research before final prompt
```

Then advanced controls:

```
Source depth: quick / balanced / deep
Allowed domains
Blocked domains
Source types
Max sources
Max token budget
Apply findings: this run only / suggest template update / auto-draft PR
```

Popup after research

```
Research found 6 possible improvements.

1. Add rate-limit handling
2. Add provider fallback
3. Add environment variable safety
4. Add deployment acceptance test
5. Add pricing/cost warning
6. Add context-window limit

Choose:
[Apply all]
```

```
[Review one by one]
[Apply high-confidence only]
[Ignore]
```

My recommendation

Yes, build it — but not before the basic Mode Manager is mounted.

Order should be:

```
1. Finish Mode Manager UI
2. Add Context Pack
3. Add Template Registry
4. Add Manual Loop Mode
5. Add Research-Augmented Prompt Mode
6. Add OpenRouter/OpenAI web-search provider
7. Add template update suggestions
8. Add auto-draft PR for template changes
```

The killer feature is not just “it searches the web.” The killer feature is:

```
It checks whether your prompt template is missing something important, then upgrade
```

That is a legitimately strong differentiator for Prompt Refinery.